CSE 396 — Computer Engineering Project

MOD-02: Flame Detection Software

Detailed Technical Reference

Responsible Authors:

Name	Student ID
Mehmet Alp ATAY	220104004020
Adil Mert ERGORUN	220104004048

Autonomous Fire Detection and Suppression Rover
Group 2

Version 0.1 – April 2026

Contents

1 Module Overview **2**

2 Constants & Macros **2**

3 Data Types **2**

3.1 `flame_status_t` — Return / Error Codes . . . 3

3.2 `flame_direction_t` — Detected Flame Direction . . . 3

3.3 `flame_data_t` — Aggregated Sensor Reading . . . 3

3.4 `flame_cb_t` — Sampling Callback . . . 4

4 Public API **4**

4.1 `flame_init()` . . . 4

4.2 `flame_start()` . . . 5

4.3 `flame_read()` . . . 5

4.4 `flame_get_direction()` . . . 5

4.5 `flame_stop()` . . . 5

5 Timing & Sampling Behaviour **6**

6 Inter-Module Protocol: MOD-01 → MOD-02 → MOD-07 **6**

7 Error Handling Guide **6**

8 Integration Example **7**

9 Known Limitations & TODOs **8**

1 Module Overview

MOD-02 is the software layer that reads raw ADC values from MOD-01 (Flame Sensing HW) and produces structured, semantically meaningful flame detection results consumed by the Autonomy FSM (MOD-07). It is the only module that knows the *meaning* of IR sensor readings: it applies thresholding, determines which sensor is dominant, and assigns a directional label.

Direction	Module	Interface
Upstream (input)	MOD-01 Flame HW	`flame_hw_read_raw(uint16_t *buffer)` – populates `raw[3]`
Downstream (output)	MOD-07 Autonomy	`flame_data_t` via `flame_read()` or callback

Table 1: MOD-02 inter-module connections

Header file: `CSE_396_HW2/Header files/MOD 02/flame_detection.h`

2 Constants & Macros

Macro	Value	Rationale
`FLAME_SENSOR_COUNT`	`3`	Three KY-026 IR sensors mounted left, center, right. Mirrors `FLAME_HW_SENSOR_COUNT` from MOD-01.
`FLAME_THRESHOLD`	`500`	ADC reading (0–1023) above which a sensor is considered to be detecting a flame. Must be validated experimentally in the target environment.
`FLAME_SAMPLE_INTERVAL_MS`	`50`	Sampling period in milliseconds (20 Hz). Chosen as a balance between responsiveness and CPU load on the ESP32.
`FLAME_ADC_MAX`	`1023`	Full-scale value of the 10-bit ADC on the ESP32. Used for normalisation if needed.

Table 2: MOD-02 constants and their rationale

3 Data Types

3.1 `flame_status_t` — Return / Error Codes

Enumerator	Value	Meaning & when returned
`FLAME_OK`	`0`	Operation completed successfully. Returned by all functions on the happy path.
`FLAME_ERR_INIT`	`-1`	Hardware initialisation failed. Returned by `flame_init()` when MOD-01's `flame_hw_init()` reports an ADC configuration error. The module must not be used until a successful re-initialisation.
`FLAME_ERR_NO_SENSOR`	`-2`	One or more sensors not responding. Indicates that at least one ADC channel returns 0 or 1023 persistently, suggesting a wiring or hardware fault.
`FLAME_ERR_TIMEOUT`	`-3`	Returned by `flame_read()` when no new sample has been produced yet (e.g. first call before the first sampling cycle completes).

Table 3: `flame_status_t` enumerator definitions

3.2 `flame_direction_t` — Detected Flame Direction

Enumerator	Value	Meaning
`FLAME_DIR_NONE`	`0`	No sensor exceeded `FLAME_THRESHOLD`; no flame detected. Rover should remain in `STATE_SEARCH`.
`FLAME_DIR_LEFT`	`1`	Left sensor (`raw[0]`) has the highest reading above threshold. Rover should turn left.
`FLAME_DIR_CENTER`	`2`	Center sensor (`raw[1]`) has the highest reading. Rover is aligned; advance forward.
`FLAME_DIR_RIGHT`	`3`	Right sensor (`raw[2]`) has the highest reading. Rover should turn right.

Table 4: `flame_direction_t` enumerator definitions

Direction logic: The direction is the sensor with the *highest* ADC reading, provided that reading exceeds `FLAME_THRESHOLD`. Tie-breaking priority: center > left > right. If no sensor exceeds the threshold, direction is `FLAME_DIR_NONE` regardless of relative values.

3.3 `flame_data_t` — Aggregated Sensor Reading

This is the primary struct shared with MOD-07. It is populated on every sampling cycle.

Field	Type	Range	Description
`raw[FLAME_SENSOR_COUNT]`	`uint16_t[3]`	0–1023	Raw 10-bit ADC readings. Index 0 = left, 1 = center, 2 = right. Updated every cycle.
`max_intensity`	`uint16_t`	0–1023	Highest value among `raw[0..2]`. Proxy for flame proximity: higher = closer or brighter.
`detected`	`uint8_t`	0 or 1	Non-zero if `max_intensity > FLAME_THRESHOLD`. Primary flame-presence flag for MOD-07.
`direction`	`flame_direction_t`	enum	Dominant direction derived from `raw`. Only meaningful when `detected == 1`.
`timestamp_ms`	`uint32_t`	ms since boot	Time of reading via `millis()`. MOD-07 should reject data older than 100 ms.

Table 5: `flame_data_t` field descriptions

3.4 `flame_cb_t` — Sampling Callback

```
typedef void (*flame_cb_t)(const flame_data_t *data);
```

Listing 1: Callback typedef

Invoked *synchronously* at the end of each 50 ms sampling cycle with a read-only pointer to the freshly populated `flame_data_t`.

Thread-safety warning: The callback runs inside the sampling loop. It must return quickly and must **not** call `flame_read()`, `flame_stop()`, or any blocking function. Blocking will stall the loop and cause missed readings.

4 Public API

4.1 `flame_init()`

```
flame_status_t flame_init(void);
```

Listing 2: Signature

Pre-condition	None. Safe to call at any point, including after a failed init.
Post-condition	On `FLAME_OK`: ADC pins configured, module ready. On error: state undefined; retry.
Calls	`flame_hw_init()` from MOD-01.
Returns	`FLAME_OK` or `FLAME_ERR_INIT`.
Side effects	Configures ESP32 ADC channels. Does **not** start sampling.

4.2 `flame_start()`

```
flame_status_t flame_start(flame_cb_t cb);
```

Listing 3: Signature

Pre-condition	`flame_init()` returned `FLAME_OK`.
Parameters	`cb` – callback per sample. Pass `NULL` to use pull model only.
Post-condition	20 Hz sampling begins. Each cycle: reads ADC, computes direction, updates buffer, invokes `cb` if set.
Returns	`FLAME_OK`.
Side effects	Starts repeating timer; consumes CPU proportional to sampling rate.

4.3 `flame_read()`

```
flame_status_t flame_read(flame_data_t *out);
```

Listing 4: Signature

Pre-condition	`flame_start()` called. `out` points to valid `flame_data_t` memory.
Post-condition	`*out` populated with the latest sample.
Returns	`FLAME_OK` if data available; `FLAME_ERR_TIMEOUT` if no sample yet.
Thread safety	Safe to call from main loop while sampling runs. Single-core copy, no locking needed.

4.4 `flame_get_direction()`

```
flame_direction_t flame_get_direction(void);
```

Listing 5: Signature

Pre-condition	At least one sampling cycle completed.
Returns	Current dominant direction. Returns `FLAME_DIR_NONE` if no sample yet.
Use case	Lightweight FSM polling when only direction is needed.

4.5 `flame_stop()`

```
void flame_stop(void);
```

Listing 6: Signature

Pre-condition	None. Safe to call even if not started.
Post-condition	Sampling halted. Sensors powered down. Last sample retained in buffer.
Side effects	Pending callback cancelled. Must call `flame_init()` + `flame_start()` to resume.

5 Timing & Sampling Behaviour

Parameter	Value / Notes
Sampling interval	50 ms (20 Hz) – `FLAME_SAMPLE_INTERVAL_MS`
ADC conversion time	$\approx$ 5 µs per channel on ESP32; three channels $\approx$ 15 µs total, negligible vs. interval.
Callback deadline	Callback must return within $\ll$ 50 ms to avoid missing the next cycle.
Data staleness limit	MOD-07 should reject `flame_data_t` with `timestamp_ms` older than 100 ms (2 missed cycles).

Table 6: MOD-02 timing parameters

6 Inter-Module Protocol: MOD-01 → MOD-02 → MOD-07

Step 1. Hardware read. MOD-02 calls `flame_hw_read_raw(raw)` (MOD-01). Three 10-bit ADC values written into `raw[0..2]`: left, center, right.

Step 2. Threshold check. For each `raw[i]`, compare to `FLAME_THRESHOLD = 500`.

Step 3. Max intensity. `max_intensity = max(raw[0], raw[1], raw[2])`.

Step 4. Detection flag. `detected = (max_intensity > FLAME_THRESHOLD) ? 1 : 0`.

Step 5. Direction. Index with highest `raw[i]` exceeding threshold → `0=LEFT, 1=CENTER, 2=RIGHT`. If none exceed threshold → `FLAME_DIR_NONE`.

Step 6. Timestamp. `timestamp_ms = millis()`.

Step 7. Publish. Buffer updated. Callback invoked (if registered). MOD-07 calls `flame_read()` or reads callback arg.

7 Error Handling Guide

Error code	Recommended action
`FLAME_ERR_INIT`	Wait 100 ms, retry `flame_init()`. After 3 consecutive failures, halt and signal MOD-08 system fault.
`FLAME_ERR_NO_SENSOR`	Check ADC wiring. Treat as hardware fault; do not continue operating with faulty sensor.
`FLAME_ERR_TIMEOUT`	Retry after `FLAME_SAMPLE_INTERVAL_MS + 10` ms. If persistent, call `flame_stop()` then `flame_start()` to restart the loop.

Table 7: MOD-02 error handling recommendations

8 Integration Example

```
#include "flame_detection.h"
#include <Arduino.h>

/* Callback: invoked by MOD-02 every 50 ms */
void on_flame_sample(const flame_data_t *data) {
    if (!data->detected) return;

    switch (data->direction) {
        case FLAME_DIR_LEFT:
            Serial.printf("[%lu ms] Flame LEFT  | intensity=%u\n",
                          data->timestamp_ms, data->max_intensity);
            break;
        case FLAME_DIR_CENTER:
            Serial.printf("[%lu ms] Flame AHEAD | intensity=%u\n",
                          data->timestamp_ms, data->max_intensity);
            break;
        case FLAME_DIR_RIGHT:
            Serial.printf("[%lu ms] Flame RIGHT | intensity=%u\n",
                          data->timestamp_ms, data->max_intensity);
            break;
        default: break;
    }
}

void setup() {
    Serial.begin(115200);

    /* Step 1: initialise hardware (MOD-01 ADC pins) */
    if (flame_init() != FLAME_OK) {
        Serial.println("ERROR: flame_init failed. Halting.");
        while (1);
    }

    /* Step 2: start 20 Hz sampling with callback */
    flame_start(on_flame_sample);
}
```

```

void loop() {
    /* Pull model: read latest data every 100 ms */
    flame_data_t latest;
    if (flame_read(&latest) == FLAME_OK) {
        /* Forward to MOD-07 autonomy layer:                          */
        /* autonomy_input_t in = { .flame_data = latest, ... };     */
        /* autonomy_update(&in);                                      */
    }
    delay(100);
}
```

Listing 7: Full MOD-02 integration with callback and error handling

9 Known Limitations & TODOs

- **Ambient light interference.** Strong sunlight or incandescent lamps generate IR radiation indistinguishable from flame by the KY-026. Shielded housings and directional tubes are recommended. `FLAME_THRESHOLD` must be tuned in the deployment environment.
- **Single-axis detection only.** Left/center/right; no elevation angle. A flame significantly above or below the sensor plane may not be detected.
- **Winner-takes-all direction.** A weighted angular average of all `raw[i]` values would provide finer directional precision than the current dominant-sensor approach.
- **Callback thread safety.** If FreeRTOS tasks are used, a mutex must protect the shared `flame_data_t` buffer.
- **TODO:** Moving-average filter over last N samples before threshold comparison to reduce noise.
- **TODO:** Add `FLAME_ERR_STALE` when `timestamp_ms` has not updated within 2× `FLAME_SAMPLE_INTERVAL_MS`.
- **TODO:** Validate `FLAME_THRESHOLD = 500` experimentally with the KY-026 at typical operating distances and lighting conditions.